

Shared Interface Contract

Read this first — every track brief assumes you’ve read this document.

This is the one file all four human+agent pairs must agree on *before* writing code. It exists so that four teams can build four different pieces of the same pipeline **at the same time**, without waiting on each other, by building against a fixed contract instead of against each other’s in-progress code. If a track deviates from this contract, integration on Day 5 breaks. If you must change something in here, post it to the group first — do not silently change a shape, a unit, or a field name.

1. The Pipeline (what we’re building)

[Photo(s)]

-> Track A: Depth Estimation & Camera Geometry

-> depth map + intrinsics estimate

-> Track B: Point Cloud & Mesh Reconstruction

-> point cloud + mesh (.ply/.obj)

-> Track C: Calibration, Scale & Backend API

-> metric-scaled mesh + job orchestration

-> Track D: Frontend Viewer, Validation & Demo

-> 3D viewer + accuracy report + demo

Each arrow above is a **file-based handoff**. No track calls another track’s Python functions directly — everyone reads/writes files (or, for Track C/D, JSON over HTTP) in the formats fixed below. This is what makes parallel work possible: everyone can fake the file they don’t own yet (see §4, Mocks) and swap in the real thing later without changing their own code.

2. Fixed data formats

Artifact	Format	Producer	Consumer
Input image	.jpg / .png , any resolution, EXIF stripped	User upload	Track A
Depth map	.npy , float32 , shape (H, W) , values = relative inverse-or-metric depth per Depth Anything V2’s native output (not yet scaled)	Track A	Track B
Camera intrinsics	intrinsics.json : {"fx": float, "fy": float, "cx": float, "cy": float, "width": int, "height": int, "estimated_fov_deg": float}	Track A	Track B, Track C
Point cloud	.ply , ASCII, fields x y z r g b , in arbitrary (unscaled) units	Track B	Track C
Mesh	.obj or .ply , triangulated, single object, arbitrary units	Track B	Track C
Scale factor	scale.json : {"scale_factor": float, "reference_method": string, "units": "meters"}	Track C	Track D
Final scaled mesh	same mesh file + scale.json applied,	Track C	Track D

	units = meters		
Job status	JSON over GET /jobs/{id}: {"id": str, "status": "queued" "running" "done" "failed", "progress": 0-100, "result_url": str null, "error": str null}	Track C	Track D
Accuracy record	accuracy_log.json, one row per test image: {"image_id", "measured", "ground_truth", "pct_error"}	Track D	Everyone (report)

Units rule: nothing is in meters until *after* Track C’s calibration step. Tracks A and B work in arbitrary/relative units on purpose — don’t try to guess real-world scale early, it just gets thrown away and recomputed.

Naming rule: every file for a given input photo shares one job ID, e.g. `job_ab12/depth.npy`, `job_ab12/intrinsics.json`, `job_ab12/mesh.obj`, `job_ab12/scale.json`. This is the only thing that lets four independently built pieces snap together on Day 5 without a rewrite.

3. Repo / folder layout

```
/project
/track_a_depth      <- Track A owns this folder only
/track_b_mesh       <- Track B owns this folder only
/track_c_api        <- Track C owns this folder only
/track_d_frontend   <- Track D owns this folder only
/shared
  /sample_jobs      <- shared test fixtures, see §4
  /schemas          <- copies of the JSON shapes above, as .json examples
/integration        <- nobody touches until Day 5 (integration day)
```

Nobody edits another track’s folder. If you need something changed in another track’s output format, that’s a message to the group, not a pull request into their folder.

4. Mocks — how to build without waiting

Each track has something it *would* need from another track before it can start. Instead of waiting, use the fixed-format fixtures in `/shared/sample_jobs/`, which are hand-made fake outputs matching §2 exactly:

- `sample_jobs/job_0001/depth.npy` + `intrinsics.json` — a fake depth map (a gradient + a few blobs) so **Track B can start Day 1** without a working depth model.
- `sample_jobs/job_0001/mesh.obj` — a simple fake mesh (a cube + a ramp) so **Track C can start Day 1** without a working reconstruction.
- `sample_jobs/job_0001/mesh.obj` + `scale.json` + a fake `/jobs/{id}` response — so **Track D can start Day 1** without a working API.
- Track A has nothing upstream (input is just a photo), so Track A starts from real data on Day 1 by default.

Human+agent pair on Track A: your first task Day 1 is generating these mock fixtures for everyone else, *before* starting your own real work. This unblocks the other three tracks immediately.

5. Schedule (5 working days, 4 tracks in parallel)



Day	All tracks
1	Build mocks (Track A), then everyone builds against mocks/fixed data in isolation
2-3	Core build, each track in its own folder, contract format frozen
4	Each track finishes, tests against the mock fixtures one more time, writes a 1-paragraph “what I actually built vs. the contract” note
5	Integration day. Real outputs get piped end-to-end, <code>/integration</code> folder, all four pairs in the room together

Day 5 is not “buffer” — budget it explicitly. This is where format mismatches surface no matter how careful everyone was, and it needs all four pairs present to debug together.

6. Definition of done (MVP-wide)

- One real photo goes in, a scaled, viewable 3D mesh comes out, end to end, through the real API (not mocks).
- At least 3 of the 5-10 hero test images run through the full pipeline and land in `accuracy_log.json`.
- The frontend shows the mesh and the measured-vs-ground-truth numbers.
- Nothing in this list requires the accuracy-upgrade backlog from the original 4-week plan — that’s explicitly out of scope for MVP week 1.